/codifica

Formação Desenvolvedor Full Stack Júnior

LISTA DE EXERCÍCIOS

Sistema de Biblioteca

Programação Orientada a Objetos (Java) + Banco de Dados (PostgreSQL)

ROTEIRO	3 exercícios
1. Java (POO)	Sistema de biblioteca com classes, herança e polimorfismo
2. Modelagem SQL	Tabelas, chaves e script de criação no PostgreSQL
3. Consultas SQL	4 consultas sobre os dados modelados no exercício 2

EXERCÍCIO 1 — Sistema de Biblioteca em Java

Uma biblioteca de bairro quer controlar seu acervo — livros e revistas — emprestado a alunos e professores. Livro: 14 dias de prazo, multa de R$ 0,50/dia. Revista: 7 dias, R$ 1,00/dia. Aluno pode ter até 3 itens simultâneos; professor, até 5.

O que fazer

3. classe abstrata ItemBiblioteca (codigo, titulo, disponivel) com prazo e multa como métodos abstratos, implementados em Livro e Revista.
4. classe abstrata Usuario (nome, quantidadeEmprestada) com limite de itens como método abstrato, implementado em Aluno e Professor.
5. classe Biblioteca com emprestar(), devolver() e listarAcervo(), usando arrays de tamanho fixo.
6. classe Main com um cenário de teste: cadastro, um empréstimo bem-sucedido e um recusado por limite atingido.

💡 Dicas para desenvolver

7. Prazo e multa variam por tipo de item — por isso são métodos abstratos, não atributos comuns.
8. Não crie um setDisponivel() público: a disponibilidade muda como consequência de emprestar() e devolver().
9. listarAcervo() deve percorrer ItemBiblioteca[] em um único laço, sem nenhum if verificando o tipo do item.

10. Teste sua solução criando uma categoria nova (por exemplo, DVD) sem alterar nenhum método já escrito — se precisar alterar, revise onde a decisão de tipo vazou.

EXERCÍCIO 2 — Modelagem do Banco de Dados

O mesmo cenário do Exercício 1, agora modelado como tabelas relacionais em PostgreSQL — sem herança: um item é "livro" ou "revista" por uma coluna tipo, e o mesmo vale para o usuário.

O que fazer

11. tabela item: id, codigo, titulo, tipo, autor, edicao, disponivel.
12. tabela usuario: id, nome, tipo, limite_itens.
13. tabela emprestimo: id, item_id, usuario_id, data_retirada, data_devolucao_prevista, data_devolucao, valor_multa.
14. escreva o script CREATE TABLE das três tabelas, na ordem correta.
15. insira dados de teste: pelo menos 4 itens, 2 usuários e 2 empréstimos (um em aberto, um já devolvido).

💡 Dicas para desenvolver

16. Use GENERATED ALWAYS AS IDENTITY para as chaves primárias — é a forma moderna do PostgreSQL, evite SERIAL.
17. Use CHECK (tipo IN (...)) para restringir os valores possíveis de tipo, tanto em item quanto em usuario.
18. item_id e usuario_id em emprestimo são chaves estrangeiras: REFERENCES item(id) e REFERENCES usuario(id).
19. data_devolucao não deve ter NOT NULL — ela fica vazia enquanto o item não voltar.
20. A ordem dos CREATE TABLE importa: emprestimo depende de item e usuario já existirem.

EXERCÍCIO 3 — Consultas SQL

Com as tabelas e os dados do Exercício 2 prontos, escreva as quatro consultas abaixo.

O que fazer

1. listar todo o acervo, com código, título, tipo e disponibilidade.

2. listar os empréstimos em aberto, com o nome do usuário e o título do item.
3. calcular o total de multas acumuladas por usuário.
4. listar os itens que nunca foram emprestados.

> 💡 Dicas para desenvolver
>
> 21. As consultas 1 e 2 precisam de JOIN para reunir dados de tabelas diferentes na mesma linha de resultado.
> 22. Um empréstimo está em aberto quando data_devolucao IS NULL — nunca compare com = NULL.
> 23. A consulta 3 pede GROUP BY (por usuário) junto com SUM(valor_multa).
> 24. A consulta 4 é a mais sutil: use LEFT JOIN entre item e emprestimo, e teste IS NULL numa coluna do lado de emprestimo — é isso que identifica quem nunca apareceu do outro lado da junção.

Ordem sugerida: resolva o Exercício 1 (Java) e o Exercício 2 (modelagem) de forma independente; o Exercício 3 depende dos dados criados no Exercício 2.